

App Development Project Report

App Dev Project Report

1. Student Details

Name: Dheepak D

Roll Number: 23F2001453

Email: 23f2001453@ds.study.iitm.ac.in

About Me: I bridge Data Science and ECE through scalable backend projects like "HMS", a full-stack Hospital Management System. Currently pursuing degrees from IIT Madras and Sri Sairam Engineering College, I focus on integrating distributed systems with Embedded Systems and Robotics

2. Project Details

Project Title: Hospital Management System V2 - Medicare.

Problem Statement:

To design and build a web-based Hospital Management System (HMS) that allows Admins, Doctors, and Patients to efficiently manage and track patient records, appointments, and treatment schedules according to their specific roles and to give timely remainder of the appointments for the patients and monthly report for the doctor.

Approach:

- The application follows a decoupled architecture a Flask REST API backend and a Vue.js SPA application frontend communicating exclusively via JSON. The backend was designed as a REST API using Flask and Flask-RESTful, with each resource represented as a Python class with HTTP method handlers (get, post, put, delete).
- Flask-Security-Too handles authentication using token-based auth — users log in once, receive a token, and include it in every subsequent request header. SQLAlchemy ORM maps Python model classes to SQLite database tables. Redis serves two purposes: as a Celery message broker for background task queuing, and as an API cache layer to reduce repeated database queries.
- Celery manages three types of jobs: daily appointment reminder emails (scheduled via Celery Beat at 08:00), monthly doctor activity reports (1st of each month), and user-triggered CSV exports of treatment history. MailHog is used as a local fake SMTP server during development, catching all outgoing emails and displaying them at localhost:8025.
- The Vue.js frontend uses Vue Router for navigation with route guards, Pinia for global

state management, and a centralised api.ts layer for all HTTP communication. Role-based access control enforces that patients, doctors, and admins each see only what is relevant to their role, both in the UI and at the API level.

3. AI/LLM Declaration

I used an AI/LLM (Chat-GPT) to assist in generating frontend CSS styling, suggesting the redis server, setting up celery workers and providing suggestions for repetitive codes for API endpoints from the frontend part

The extent of AI/LLM usage is around ~15-20%, limited to aesthetic suggestions, ~10-15% for model logic and boilerplate code.

All final implementation logic, including the database interactions (CRUD operations), authentication/authorization setup, and integration of the three user roles (Admin, Doctor, Patient), and the mail functionalities were done manually.

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend web framework — routes, app factory, config
Flask-RESTful	REST API structure — Resource classes with HTTP method handlers
Flask-Security-Too	Token-based authentication, password hashing, role management
Flask-SQLAlchemy	ORM — maps Python classes to SQLite database table
Flask-Mailman	Email sending — connects to MailHog SMTP for development
Flask-CORS	Allows Vue.js frontend (5173) to call Flask API (port 5000)
SQLite	Lightweight relational database for storing all application data
Redis	In-memory store — Celery broker (DB 0) and API cache (DB 1)
Celery	Distributed task queue for background and scheduled jobs
Celery Beat	Scheduler that triggers daily reminders and monthly reports
Flower	Real-time web UI for monitoring Celery workers and tasks
MailHog	Fake SMTP server — catches emails locally at localhost:8025
Vue.js 3	Frontend SPA framework with Composition API (script setup)
Vue Router	Client-side routing with navigation guards for role protection

Pinia	Global state management — auth store shared across all components
Docker / Docker Compose	Containerises Redis and MailHog for consistent local dev environment

5. Database Schema / ER Diagram

The Hospital Management System (HMS) application utilizes seven main database models to manage user roles, appointments, medical records, and hospital resources:

- 1. User:** Stores core user details and authentication info for all roles (Admin, Doctor, Patient), including the primary key id, username, email, password_hash, and the essential role field.
- 2. Department:** Stores hospital department information (e.g., Cardiology, Orthopedics), identified by its id, name, and description.
- 3. Doctor:** Stores Doctor-specific details, linked via a one-to-one relationship to the User table (user_id). Key fields include specialization and department_id.
- 4. Patient:** Stores Patient-specific details, linked via a one-to-one relationship to the User table (user_id). It includes fields like emergency_contact.
- 5. DoctorAvailability:** Stores specific time slots and schedules when a doctor is available. It links to the Doctor via doctor_id and includes fields like date, start_time, end_time, and slot_capacity.
- 6. Appointment:** Stores scheduled appointments, which link a Patient (patient_id), a Doctor (doctor_id), and a specific DoctorAvailability slot (availability_id), along with the date, time, and status.
- 7. Treatment:** Stores the medical record or notes for a specific appointment, linked via a one-to-one relationship to the Appointment table (appointment_id). Fields include diagnosis, prescription, and treatment_notes.

Relationships:

- One-to-One: User → Doctor
- One-to-One: User → Patient
- One-to-One: Appointment → Treatment
- One-to-Many: Department → Doctor
- One-to-Many: Doctor → DoctorAvailability
- One-to-Many: Doctor → Appointment
- One-to-Many: Patient → Appointment
- One-to-Many: DoctorAvailability → Appointment

6. API Resource Endpoints

1) Authentication:

Method	Endpoint	Description
POST	/api/auth/register	Register a new patient or doctor account
POST	/api/auth/login	Authenticate and receive auth token
POST	/api/auth/logout	Authenticate and receive auth token
GET	/api/auth/me	Authenticate and receive auth token
PUT	/api/auth/profile	Update own profile fields and password

2) Doctor - Availability Management

Method	Endpoint	Description
GET	/api/doctor/availability	List own availability slots
POST	/api/doctor/availability	Create a new time slot
PUT	/api/doctor/availability/	Edit an existing slot
DELETE	/api/doctor/availability/	Delete a slot (if no active bookings)

3) Doctor - Appointments and Patients:

Method	Endpoint	Description
GET	/api/doctor/appointments	List all appointments assigned to doctor
POST	/api/doctor/appointments	Advance appointment status
GET	/api/doctor/patients	List all unique patients seen
GET	/api/doctor/patients//history	Full appointment history for a patient

POST	/api/doctor/appointments	Save diagnosis and prescription
PUT	/api/doctor/appointments	Update an existing treatment record

4) Patient - Browsing and Booking:

Method	Endpoint	Description
GET	/api/doctors/	List all doctors (with filter support)
GET	/api/doctors//availability	View open slots for a doctor
POST	/api/appointments/book/	Book a specific availability slot
GET	/api/appointments/my	View own appointments
POST	/api/appointments//cancel	Cancel a booked appointment
GET	/api/appointments//treatment	View own treatment/prescription record

5) Patient - Export:

POST	/api/patient/export-csv	Trigger async CSV export of treatment history
GET	/api/jobs/	Poll Celery task status

6) Admin:

Method	Endpoint	Description
GET	/api/admin/users	List all users (paginated, filterable)
GET	/api/admin/users	Get a specific user's profile
PUT	/api/admin/users	Edit any user's details
DELETE	/api/admin/users	Permanently delete a user
POST	/api/admin/users//blacklist	Blacklist a user (invalidates session)
DELETE	/api/admin/users//blacklist	Restore a blacklisted user

GET	/api/admin/appointments	View all appointments system-wide
GET	/api/admin/appointments//treatment	View any treatment record
GET	/api/admin/search	Search users by name, email, phone, specialization

7. Architecture and Features (optional)

Architecture Overview:

1. Vue.js 3 + Vite – Decoupled frontend SPA with Pinia state management and centralized API handling.
2. Flask + Flask-RESTful – REST API server utilizing Resource classes, auth guards, and JSON responses.
3. Flask-Security-Too – Token-based authentication with RBAC, bcrypt hashing, and role management.
4. SQLAlchemy + SQLite – ORM layer featuring 8 model classes with defined relationships and cascade deletes.
5. Celery + Redis – Background job processing and scheduled tasks via Celery Worker and Beat.
6. Redis Caching – Performance optimization through API response caching with variable TTL settings.
7. Flask-Mailman + MailHog – SMTP email integration with local mail capture for testing.
8. Flower – Real-time monitoring dashboard for tracking Celery workers and task history.

Implemented Features:

- Fully decoupled frontend/backend architecture
- Secure token-based authentication and Role-Based Access Control (RBAC)
- Asynchronous background job processing and task scheduling
- Performance-driven API response caching
- Real-time monitoring of background workers and tasks
- Relational database management with automated cascade operations
- Integrated email notification system with local debugging
- Centralized frontend state management and routing

8. Video Presentation

Drive Link:

 [MAD_2.mp4](#)